Project Phase 2 Report

CITS5501/CITS3501 Software Testing and Quality Assurance, Semester 2 2026

Name	Student number	GitHub ID
Rivin Sharma	24295598	rivinsharma
Wenkow Sun	24168584	wenkow2k515
Shravan Suresh Kumar	24975803	ShravansKumar
Kennice Leong Jing Lin	24271935	kenniceleongg
Nathan Lance Macaspac Buluran	23402727	naetnaetnaet

Repository: github.com/wenkow2k515/CITS5501-3501-Group-Project

1. Task 1 – Prototype

1.1 Overview

Our prototype, scraper.py, is a Python script built on the supplied starter code. It uses the `requests` library for HTTP and BeautifulSoup to parse HTML. By default it starts from the policy index page; an alternative start URL can be given as a command-line argument, so the same script can be run against a local copy of the site during testing.

The scraper works in four stages:

1. **Fetch the index page.** The HTTP status and declared content type are printed.
2. **Extract links.** Every `<a href>` element is collected and resolved to an absolute URL against the final URL of the index response.
3. **Select policy documents.** A link is treated as a policy document if its path ends in a document extension (`.pdf`, `.doc`, `.docx`, `.xls`, `.xlsx`, `.ppt`, `.pptx`, `.rtf` or `.odt`). The check is case-insensitive and ignores query strings. Links to other hosts are excluded, and URL fragments are removed before duplicate URLs are discarded. Different URLs may still refer to identical document contents.
4. **Download and report.**The scraper attempts to download the first three distinct document URLs. For each, the scraper prints the URL, HTTP status, declared `Content-Type`, downloaded size in bytes and the first 10 bytes of the body, so the `%PDF` or `PK` file signature can be checked. The downloaded data is then discarded.

All requests go through the starter's `fetch()` function, which enforces a 10-second timeout, waits 1 second between fetch calls, and a limit of 20 fetch calls per run. Automatic redirect requests are not counted separately, therefore a typical run without redirects makes four HTTP requests consisting of the index page and 3 documents

1.2 Limitations

Because the prototype is intentionally minimal, it has several limitations:

- **Success is judged by status code only.** Any `200 OK` response is reported as a successful download, so a `.pdf` link that returns an HTML page, or a corrupt PDF, would still be reported as successful. The first bytes are printed but not checked against the expected file type; the artefact in section 3.4 demonstrates this failure.
- **Documents are identified by file extension only.** Documents served from URLs without an extension (for example `/download?id=12`) are missed, and any URL ending in a document extension is assumed to be a document.
- **Network errors are not handled.** A timeout or connection error raises an unhandled exception and ends the run, rather than being reported for that document while the remaining downloads continue. The brief does not require this for the prototype, but a production scraper would need it (see tests T005 and T006).
- **Only static links are found.** Links generated by JavaScript or held in markup other than `<a href>` are not discovered, and the scraper cannot distinguish an index page with no documents from one whose structure has changed.
- **Responses are read fully into memory.** Each response is read in one go rather than streamed, so a very large file could exhaust the available memory.

1.3 Assumptions

To keep the prototype within the scope of the brief, we made the following assumptions:

- All policy documents are linked directly from the index page, so no recursive crawling is needed, as permitted by the brief.
- Policy documents are served from the same host as the index page, so links to other hosts can be ignored.
- Downloading three distinct documents is enough to demonstrate the approach, and stays well within the 20-request limit.
- Relative links are resolved against the final URL of the index response, after any redirects.

2. Task 2 – Design of test suite

A production scraper targets one known site, so in most of these scenarios it cannot recover by itself. The reasonable expectation is that it fails predictably and visibly. Across all scenarios, a well-engineered scraper should:

- never terminate with an unhandled exception;
- never report a failed or invalid download as successful;
- report what went wrong, and for which URL;
- continue with the remaining documents where it is safe to do so;
- stay within its request limit, including any retries.

2.1 Expected behaviour per scenario

1. **Website structure changes.** If policy links are still ordinary `<a href>` elements, they should still be found. If the page loads but no policy links are found, the scraper

should report a probable structure change and exit with an error, rather than finishing successfully with no results.

2. **Truncated HTML.** The scraper should not crash. Complete links should still be processed and a partial link should not be requested. A warning should be required when there is evidence of an interrupted transfer, such as fewer bytes arriving than the declared content length. A missing closing </html> tag alone does not establish truncation
3. **Timeouts (short or long).** Every request needs a timeout. If the index page times out, nothing further can be done, so the scraper should report this and stop. If a single document times out, it should be reported as failed and the scraper should move on. A short outage may justify a limited retry, provided retries count towards the request limit.
4. **4xx and 5xx responses.** The status code should be checked before the response body is used. An error page returned for the index must not be parsed as the index, and an error response for a document must never be counted as a downloaded document.
5. **Malformed Office or PDF files.** The file extension should not be trusted. The declared `Content-Type` and the file signature (`%PDF` for PDF, `PK` for Office Open XML formats such as `.docx`) should be compared with what was expected, and a mismatched or unreadable file reported as a failure.

2.2 Test cases

All tests run against a local copy of the site (saved HTML and sample documents served from `localhost`) or use a test double in place of the network, so they are repeatable and do not depend on the live site. Task 3 describes these fixtures and doubles. A test fails if the scraper crashes with a traceback, reports a failed or invalid document as successfully downloaded, or makes more than 20 requests.

Table 1. Test cases for the five failure scenarios, with one baseline case.

ID	Scenario	Arrange	Act	Assert
T001	Baseline (normal operation)	Serve a saved copy of the policy index page locally, linking to three known valid PDF and DOCX files.	Run the scraper on the local index URL.	Exactly the expected policy links are found. Each download reports status 200, the expected Content-Type, a non-zero size and the correct signature (%PDF or PK). Exit code is 0.
T002	1. Structure change (links still present)	Modify the fixture index so policy links sit in different markup: moved from a list into a table, wrapped in extra elements, and relative instead of absolute.	Run the scraper on the modified page.	The same documents as in T001 are found and downloaded.
T003	1. Structure change (links no longer found)	Modify the fixture index so documents are no longer plain <a href> links, for example loaded by JavaScript or behind onclick buttons. The page still returns 200.	Run the scraper on the modified page.	The scraper reports that no policy documents were found and that the structure may have changed, and exits with a non-zero code.

ID	Scenario	Arrange	Act	Assert
T004	2. Truncated HTML	Serve index which contains two complete links, /a.pdf and /b.pdf followed by unfinished text <a href “/partial.pdf , with no closing quote or angle bracket.	Run the scraper on the truncated page.	No crash. Only /a.pdf, /b.pdf requested and /partial.pdf is not. Warning optional for HTML only fixture because the HTTP transfer itself may be complete.
T005	3. Index page timeout (long outage)	Use a test double that raises a timeout for the index request, or a local server that delays longer than the scraper's timeout.	Run the scraper.	Within roughly the configured timeout, the scraper reports that the index page could not be reached and exits with a non-zero code. No traceback and no document requests.
T006	3. Document timeout (short outage)	The index is normal; one document URL times out and the others respond normally.	Run the scraper.	The timed-out document is reported as failed with the reason, and the remaining documents are still downloaded. Total requests, including any retry, stay at or below 20.
T007	4. Index returns 5xx	Make the index URL return 503 Service Unavailable with an HTML error page.	Run the scraper.	The scraper reports the 503 and stops. No links are extracted or followed from the error page.
T008	4. Document returns 4xx	The index is normal; one policy link returns 404 Not Found.	Run the scraper.	That document is reported with status 404 and counted as failed. Other documents are unaffected.
T009	5. PDF link returns HTML	One link ending in .pdf returns 200 OK with Content-Type text/html and an HTML body.	Run the scraper.	The document is flagged as not a valid PDF (content type mismatch, and the body does not start with %PDF) and is not counted as a successful download.
T010	5. Corrupt PDF	One link returns 200 OK and Content-Type application/pdf, with a body that starts with %PDF but is truncated after the header (no %%EOF marker).	Run the scraper.	The document is reported as invalid. Checking the signature alone passes this file, so the test needs a deeper check such as opening it with a PDF library. Our Task 1 prototype would fail this test.

3. Task 3 – Fixtures and test doubles

3.1 Fixtures as a source of truth

Each test compares the scraper's output with a fixture whose correct result is known in advance. We would use:

- **Saved HTML pages.** A copy of the policy index page captured once from the live site and stored in the repository, plus variants derived from it: restructured markup, a truncated copy and a copy with the links removed (T002 to T004). For each page we record the exact list of document URLs that should be found.
- **Sample policy files.** Known-good PDF and DOCX files, and deliberately broken ones: an HTML page saved under a `.pdf` name and a PDF cut off after its header

(T009, T010). Each file is stored with its expected status, content type, size and signature.
- **A local HTTP server.** A small server, started by the tests, that serves these pages and files from `localhost` and can be configured to return chosen status codes, headers or delays (T005 to T008).

Because the fixtures only change when we change them, a failing test points to the scraper rather than to the site. The captured index page should still be refreshed occasionally and compared with the live site by hand, so the fixtures stay representative.

3.2 Request limits on the target site

We probed the site with curl from one machine, identifying the requests with our scraper's User-Agent name. A `HEAD` request for one policy PDF returned `200 OK` with `Content-Type: application/pdf` and a `Content-Length` of 163,909 bytes, together with the headers `X-Ratelimit-Limit: 3`, `X-Ratelimit-Remaining: 2`, `X-Ratelimit-Reset` and `Retry-After: 10`. The site is served through Cloudflare.

We then downloaded the same PDF ten times in a row with no delay between requests (each took about 0.4 seconds). The first six requests returned `200 OK` with the full 163,909 bytes. Requests seven to ten returned `429 Too Many Requests` with a 40,740-byte error page instead of the document.

The site therefore does limit requests. The headers advertise a limit of three, yet six back-to-back downloads succeeded before the first 429, so the exact limit and window are not stated; we did not probe further, to stay within the brief's limit of ten downloads. This affects testing in two ways: a scraper has to space its requests and honour `Retry-After`, and a 429 response must never be counted as a downloaded document, even though its size looks plausible.

Because the live site limits requests, the Task 2 tests should not run against it at all. They run against the fixtures and test doubles described in 3.1 and 3.3, which allows any number of runs, including in CI. The live site is then used only for an occasional smoke test that stays well within its limit, and to refresh the captured fixtures.

3.3 Simulating network failures with test doubles

Test doubles can simulate timeouts, interrupted downloads, connection failures and very slow responses without touching the live site. In Python this can be done at three levels:

- **Replacing the HTTP call.** `unittest.mock.patch` or pytest's `monkeypatch` can replace `fetch()` or `requests.Session.get` with a double that raises `requests.exceptions.Timeout`, `ConnectionError` or `ChunkedEncodingError` (an interrupted download). The `responses` and `requests-mock` libraries do the same per URL, returning chosen status codes, headers and bodies. These doubles are fast and deterministic: a timeout test raises the exception immediately instead of waiting for the real 10-second limit.
- **A real local server.** `pytest-httpserver`, or Python's built-in `http.server` running in a thread, can delay a response beyond the client timeout, send data a few bytes at

a time, or close the connection part-way through a body whose `Content-Length` promises more. These cases depend on real socket behaviour, so a mock of `requests` cannot reproduce them faithfully.
- **Network-level faults.** Toxiproxy runs between the scraper and a server and adds latency, bandwidth limits or connection resets, which suits tests of very slow or unstable connections.

3.4 Demonstration artefact

Our artefact, fixtures/fake_pdf_server.py, demonstrates Task 2 scenario 5 (tests T009 and T010). It is a local server, written with only the Python standard library, that serves a policy index linking to three documents:

- `valid.pdf`: a minimal well-formed PDF served as `application/pdf`;
- `not-a-pdf.pdf`: an HTML error page served with `200 OK` and `Content-Type: text/html`;
- `corrupt.pdf`: a 40-byte file that starts with `%PDF` but is truncated, with no `%%EOF` marker.

Run against this server, the Task 1 prototype reports all three documents as "Downloaded successfully", although the second is an HTML page and the third cannot be opened by a PDF viewer. tests/test_fake_pdf.py automates the check: the valid-PDF test passes and the other two fail, which is the failure the fixture was built to expose. Both files are in the submitted zip, and README.md explains how to run them.

The same scenario is difficult and undesirable to reproduce against the live site:

- we do not control the live server, so we cannot make it return HTML or a corrupt file for a PDF link on demand;
- even if a broken document existed there, it could be fixed or removed at any time, so the test would not be repeatable and its expected result would not be known in advance;
- every run would use requests on a shared, rate-limited site, and would fail whenever the site or network was unavailable, for reasons unrelated to the scraper.

4. Task 4 – Adversarial inputs

The ideas below target assumptions a scraper commonly makes about its environment, beyond the Task 2 scenarios. Each states the assumption challenged, the input or environment used to test it, and what counts as a failure and how it would be detected.

Table 2. Adversarial test ideas.

ID	Assumption challenged	Input or environment	Failure and how it is detected
A01	Document links can be identified by file extension.	Index linking to Policy.PDF (upper case), policy.pdf?v=2, a decoy policy.pdf.html, and an extensionless /download?id=12 that serves a PDF.	Decoy links are counted as policy documents, or the extensionless PDF is missed. Detected by comparing the scraper's document list with the known list for the fixture.

ID	Assumption challenged	Input or environment	Failure and how it is detected
A02	Links are well-formed, unique and use HTTP.	Index containing href="", javascript:void(0), mailto: and ../../x.pdf links, the same PDF twice with different #fragments, and percent-encoded or Unicode file names.	A document is downloaded twice, a non-HTTP link is requested or causes a crash, or a relative link resolves to the wrong URL. Detected from the distinct URLs in the local server's access log.
A03	Pages are UTF-8 and links are in static markup.	Index whose charset declaration does not match its actual encoding, and links present only in onclick handlers or inline JavaScript.	Garbled URLs, silently missed links, or a UnicodeEncodeError when printing. Detected by comparing the requested URLs in the access log with the expected set.
A04	The network path is trustworthy.	Local HTTPS server with a self-signed or expired certificate, a host that resolves to several addresses where one is unreachable, and a server that sends an invalid Content-Encoding.	The scraper crashes on the TLS or decompression error, or disables certificate verification to continue. Detected from the exit code and error output, and by checking that verification stays enabled.
A05	Links and redirects stay on the policy site.	Index linking to another host, a file:// URL and a localhost URL, plus a long redirect chain and a redirect loop.	The scraper follows off-site or local links or redirects, or loops until it reaches the request cap. Detected from the server logs and the request count.
A06	Responses are a reasonable size and arrive promptly.	A very large file (several GB), a response that never ends, and a server that sends one byte per second so that no single read times out.	Memory use grows without bound, or the run hangs or takes far longer than the configured timeout suggests. Detected by monitoring memory use and putting a time limit on the whole run.
A07	Response headers are accurate.	Server that declares a Content-Length larger or smaller than the body it actually sends, and a chunked response with no Content-Length header.	A truncated download is reported as complete and valid, or the reported size does not match what arrived. Detected by comparing the reported size with the number of bytes the server actually sent.